

Module 17: JavaScript

17.1 Fundamentals of JavaScript

Introduction of JavaScript

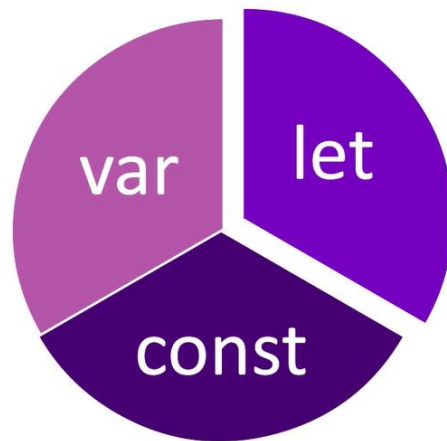
- It is an open-source programming language, well known for the development of web pages.
- It adds interactivity to the website.
- It was invented by Brendan Eich.
- It can dynamically modify an HTML page.
- JavaScript user interaction does not require any communication with the server.

Features of JavaScript

- Light-weight, versatile, Client-side scripting language
- Bottom-Up Approach
- Third-Party API's
- Wrapping Up- HTML, CSS, JS
- Manipulate HTML and CSS
- Case Sensitive

Variables in JavaScript

- Variable is name given to memory location in JavaScript.
- var – function scope
- let – block scope
- const – block scope



Difference between var, let, and const

var :

- Variable declared with var keyword has global or function/local scope.
- var can be updated or re-declared within its scope.
- var variable initialized with undefined.
- var can be declared without initialization.

let :

- Variable declared with let keyword has block scope.
- let can be updated but not re-declared .
- let variable are not initialized with undefined.
- let can be declared without initialization.

const :

- Variable declared with const keyword has block scope.
- const variable neither be updated nor re-declared.
- const variable are not initialized with undefined.
- const must be initialized during declaration.

Demo

live.com To exit full screen, press Esc

17.1 Fundamentals of JavaScript

To-Do

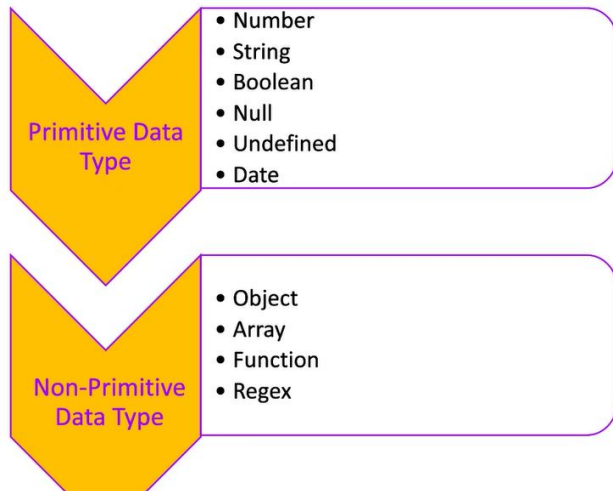
17.1.1 Variables in JavaScript



- Demonstrate the use of variable blocks in JavaScript.
- Write a program in JavaScript to store the employee information like company name, employee name, salary, mobile number, employee id and display the information except sensitive information.
- (Hint: For sensitive information(such as salary, mobile number) use block scope)

Datatypes in JavaScript

- JavaScript is a dynamically typed language.
- Variables isn't associated with a specific type.
- Types :
 - Primitive datatype –
 - ✓ It can hold a single value.
 - Non-Primitive datatype -
 - ✓ It can hold multiple values.



To-Do

17.1.2 Datatypes in JavaScript



- Demonstrate the use of primitives and objects in JavaScript.
- Write a JavaScript program to achieve below requirements:
 - a. Store five employee's information like id, name, email and display it.
 - b. Display even numbers from a given array.

Hint : a. Employee information, let emp =[{id:11, name:"Pooja", email:"pooja@gmail.com"}, {id:12, name:"Mrunal", email:"mrunal@gmail.com"},...];

b. Array is, const arr=[11,2,13,18,7,15,6,9,14];

17.2 Conditional Constructs

Block:

- if - It is used to test the true condition only.

```
<> ifBlock.html > ...
1  <html>
2    <script>
3      let empName = "Pooja";
4      let eid = "Emp-001";
5      let isExperienced = true;
6      let department = "SFDC";
7
8      if(isExperienced)
9      {
10         console.log(eid);
11         console.log(empName);
12      }
13
14    </script>
15
16  </html>
```

Block:

- if...else
- It works like yes or no.
- Display the true block or false block based on condition satisfaction.

```
<> Test.html > ...
1  <html>
2    <script>
3      let num=10;
4
5      if(num % 2 === 0)
6      {
7         console.log("Number is Even");
8      }
9      else
10     {
11        console.log("Number is Odd");
12     }
13    </script>
14
15  </html>
```

To-Do

17.2.1 Conditional Constructs



- Demonstrate when to use conditional construct blocks.
- Write a program in JavaScript to store a character and check whether that character is a Vowel or a Consonant using conditional constructs.

17.3 Functions

Description

1. It is a module which executes the block of statements.
2. It is an object and a subprogram designed to perform a specific task.
3. Gets Executed when called and returns a value.

Syntax

“function” keyword is used to write a function. Function name can contain letter, digit, underscore and dollar sign.

Types

1. Predefined
2. User-Defined

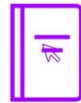
Predefined Functions

- `alert(msg)` - display message to end user.
- `prompt(msg,input)` - input dialog box.
- `eval()` - Convert a string to integer or float value.
- `isNaN(value)` - test the value is number or not.
- `parseInt(string)` - convert string into integer.
- `parseFloat(string)` – convert string into floating point.
- `setTimeout(func, milliseconds)` - execute the block once the timer expires.
- `setInterval(func, interval)` - calls a function at specified interval.
- `confirm(string)` – it will ask for continuation...

Demo

17.3 Predefined Functions

```
<> predefined.html > html > script
1  <html>
2      <script>
3          let name = prompt("Enter Your first name","Ex. Pooja");
4          alert("Hello "+name);
5          let num1 = prompt("Enter 1st Number","0");
6          let num2 = prompt("Enter 2nd Number","0");
7          let ans = eval(num1) + eval(num2);
8          ans = parseInt(ans);
9          console.log(isNaN(ans));
10         if(!(isNaN(ans)))
11         {
12             console.log("answer is :");
13             console.log(ans);
14         }
15         let test =confirm("Do you want to continue?");
16         if(test)
17         {
18             console.log("Hello User");
19         }
20         else
21         {
22             console.log("Exit!!");
23         }
24     }
25 </script>
26 </html>
```



To-Do

17.3.1 Functions



- Demonstrate the use of predefined functions
- Write a JavaScript program to accept a number and name from user and display the name to the given number of times.

Hint : If user entered number is 7 then name should be displayed 7 times.

User-Defined Functions

Function Body

```
function function_name(){  
    //function statement;  
    return;  
}
```

Program

```
//statement(s);  
alert(function_name());  
console.log(function_name());  
document.write(function_name());
```

17.4 JavaScript Exception Handling

Introduction

- Exceptions are also known as Runtime Errors.
- To handle the abnormal execution of the code JavaScript provides exception handling blocks:
 - try – try block code is executed first
 - catch – if try throws an exception then only the catch block will be executed.
 - finally – finally block will always be executed before control flow exits. It gives guarantee of execution.
 - throw – throw statement throws the user defined exception.

17.5 Working with DOM

Introduction

DOM stands for **Document Object Model**. It is a JavaScript class that defines HTML element as objects. It supports properties, methods, events.

Document Object Properties	Document Object Methods	Document Object Events
• Window Object • Document Object • Form Object	• write("string"); • getElementById() • getElementByName() • getElementByTagName() • getElementByClassName()	• onclick() • onblur() • onfocus() • onchange() • oninput() • onload() • onmouseover()

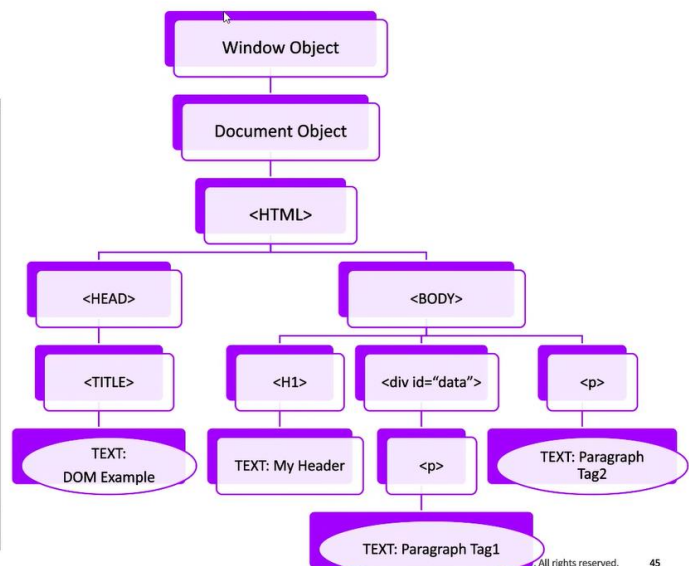
17.5 Working with DOM

Document Object Model

HTML Document

```
<> DOM.HTML > html
1  <html>
2    <head>
3      <title> DOM Example </title>
4    </head>
5
6    <body>
7      <h1> My Header </h1>
8      <div id="data">
9        <p> Paragraph Tag1 </p>
10     </div>
11     <p> Paragraph Tag2 </p>
12   </body>
13 </html>
```

Document Object Model



Methods of Document Object

- write("string") – display message on document window.
- getElementById() – It is used to read or edit an HTML element.

```
Dom_MethodDemo.html > html
1  <html>
2    <body>
3      <h1>
4        Hello User!!
5      </h1>
6      <p id="para1"> Test  </p>
7
8      <script>
9        document.writeln("Explore DOM Methods!!!");
10
11        document.getElementById("para1").innerHTML="called document.getElementById()";
12      </script>
13
14    </body>
15  </html>
```

Methods of Document Object

- getElementsByName() – it returns a collection of elements with specified name.

```
elementByName.html > html > body > script > handleClick > _len
1  <html>
2  <body>
3    <li>
4      <ol name="mobileBrands">Apple</ol>
5      <ol name="mobileBrands">Samsung</ol>
6      <ol name="mobileBrands">OnePlus</ol>
7    </li>
8
9    <input type="submit" value="Check List Length" onclick="handleClick()"/>
10
11    <script>
12      function handleClick()
13      {
14
15        let _len = document.getElementsByName("mobileBrands").length;
16        let data = "Length of List is: "+_len;
17        alert(data);
18        console.log(data);
19      }
20    </script>
21  </body>
22  </html>
```

Demo

17.6 HTML/DOM Events (Client-Side validation using JavaScript:)

Apply HTML/JS validation on client-side pages.



```
html_Dom_validation.html > html > body > script
1 <html>
2 <head>
3   <title>JavaScript Form Validation Demo</title>
4   <style type="text/css">
5     p
6     {
7       color: red;
8     }
9   </style>
10 </head>
11 <body>
12   <script type="text/javascript">
13     function validationPage()
14     {
15       const regex = /^[A-Za-z0-9]/;
16       if(regex.test(username))
17       {
18         document.getElementById("err1").innerHTML=
19           "Username should not contain special character";
20       }
21       const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
22       if(!(emailRegex.test(emailVal)))
23       {
```

17.6 HTML/DOM Events

- Output : Only HTML Validation

Demo

17.6 HTML/DOM Events (Client-Side validation using JavaScript:)

```
42 |         maxlength="25" autocomplete="off" autofocus" />
43 |     <p id="err1"></p>
44 |
45 |     <label for="email">Email:</label>
46 |     <input type="text" name="emailVal" id="emailVal" required autocomplete="off" />
47 |     <p id="err2"></p>
48 |
49 |     <label for="password">Password:</label>
50 |     <input type="password" name="password" id="password" required
51 |     minlength="8" maxlength="25" autocomplete="off" />
52 |     <p id="err3"></p>
53 |
54 |     <label for="confirm-password">Confirm Password:</label>
55 |     <input type="password" name="confirm-password" id="confirm-password"
56 |     required minlength="8"maxlength="25"autocomplete="off" />
57 |     <p id="err4"></p>
58 |
59 |     <input type="submit" value="Sign Up" onclick="validationPage()" />
60 | </form>
61 </body>
62 </html>
```

re, All rights reserve

Demo

17.6 HTML/DOM Events (Client-Side validation using JavaScript:)

```
23 |     {
24 |         document.getElementById("err2").innerHTML="Invalid Email";
25 |     }
26 |
27 |     let pass = document.getElementById("password");
28 |     let con_pass = document.getElementById("confirm-password");
29 |     if(!(pass.value === con_pass.value))
30 |     {
31 |         document.getElementById("err4").innerHTML=
32 |         "Password and Confirm-Password does not matched!!";
33 |     }
34 |
35 | }
36 </script>
37 <form id="signup">
38 <h1>Sign Up</h1>
39
40 <label for="username">Username:</label>
41 <input type="text" name="username" id="username" required minlength="8" />
```

Frontend ES 3034 Arventure. All right